

CSS Display

Complete Guide with Code Examples & Practical Usage

Introduction

The CSS display property is one of the most important properties for controlling how elements are rendered on a page. It determines whether an element is treated as inline, block, or something else entirely. Understanding display values is essential for creating modern, responsive web layouts. This comprehensive guide covers all display values, techniques, and practical applications.

Display Values Reference

Value	Description	Takes Width/Height	Margin/Padding
inline	Flows with text, no line break	No	Horizontal only
block	Full width, starts new line	Yes	All sides
inline-block	Inline but accepts dimensions	Yes	All sides
flex	Flexible layout container	Yes	All sides
grid	Grid layout container	Yes	All sides
none	Removed from document flow	N/A	N/A
table	Behaves like <table>	Yes	All sides

Example 1: Display Inline

Elements flow with text, no line breaks.

CSS Code:

```
/* Inline display */ span { display: inline; background: #e0f7fa; padding: 5px; border: 1px solid #00bcd4; } /* Characteristics */ /* - Flows with text */ /* - Ignores width/height */ /* - Horizontal margin/padding only */ /* - Multiple elements per line */
```

HTML Usage:

```
<span style="display: inline; background: #e0f7fa; padding: 5px;">Inline 1</span> <span style="display: inline; background: #e0f7fa; padding: 5px;">Inline 2</span>
```

Example 2: Display Block

Elements take full width and start new lines.

CSS Code:

```
/* Block display */ div { display: block; background: #e0f7fa; padding: 15px; border: 1px solid #00bcd4; margin: 10px 0; } /* Characteristics */ /* - Takes full width available */ /* - Starts new line */ /* - Accepts width/height */ /* - Respects margin/padding all sides */
```

HTML Usage:

```
<div style="display: block; width: 200px; padding: 15px;">Block element </div>
```

Live Demo:

Inline element 1 Inline element 2 Inline element 3 flows with text naturally

Example 3: Display Block

Elements take full width and start new lines.

CSS Code:

```
/* Block display */ div { display: block; background: #e0f7fa; padding: 15px; border: 1px solid #00bcd4; margin: 10px 0; } /* Characteristics */ /* - Takes full width available */ /* - Starts new line */ /* - Accepts width/height */ /* - Respects margin/padding all sides */
```

HTML Usage:

```
<div style="display: block; width: 200px; padding: 15px;">Block element </div>
```

Live Demo:

Block element 1 - takes full width

Block element 2 - new line

Block element 3 - another new line

Example 4: Display Inline-Block

Elements flow inline but accept width and height.

CSS Code:

```
/* Inline-block display */ div { display: inline-block; width: 120px; background: #e0f7fa; padding: 15px; border: 1px solid #00bcd4; text-align: center; } /* Characteristics */ /* - Flows inline */ /* - Accepts width/height */ /* - Multiple per line if space */ /* - Full margin/padding support */
```

HTML Usage:

```
<div style="display: inline-block; width: 100px; padding: 15px;">Inline-block </div>
```

Live Demo:

Item 1

Item 2

Item 3

Example 5: Display Flex

Flexible box layout for modern responsive design.

CSS Code:

```
/* Flex container */ .container { display: flex; gap: 10px; justify-content: space-between; } /* Flex item */ .item { flex: 1; background: #e0f7fa; padding: 15px; border: 1px solid #00bcd4; text-align: center; } /* Properties */ /* - justify-content: horizontal alignment */ /* - align-items: vertical alignment */ /* - flex-wrap: wrap items to new line */ /* - gap: space between items */
```

HTML Usage:

```
<div style="display: flex; gap: 10px;"> <div style="flex: 1;">Item 1</div> <div style="flex: 1;">Item 2</div> </div>
```

Live Demo:

Flex Item 1

Flex Item 2

Flex Item 3

Example 6: Display Grid

Powerful grid-based layouts with rows and columns.

CSS Code:

```
/* Grid container */ .container { display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px; } /* Grid item */ .item { background: #e0f7fa; padding: 20px; border: 1px solid #00bcd4; text-align: center; } /* Properties */ /* - grid-template-columns: column sizes */ /* - grid-template-rows: row sizes */ /* - gap: space between cells */
```

HTML Usage:

```
<div style="display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px;"> <div>Grid Item 1</div> <div>Grid Item 2</div> </div>
```

Live Demo:

Grid 1

Grid 2

Grid 3

Grid 4

Grid 5

Grid 6

Example 7: Display None vs Visibility Hidden

Remove elements from layout or just hide them visually.

CSS Code:

```
/* Display none - removed from flow */ .hidden-none { display: none; } /* Element takes no space */ /* Completely removed from layout */ /* Visibility hidden - hides but keeps space */ .hidden-visibility { visibility: hidden; } /* Element hidden but space reserved */ /* Layout not affected much */ /* Opacity 0 - transparent but interactive */ .hidden-opacity { opacity: 0; } /* Invisible but still clickable */ /* Takes full space */
```

HTML Usage:

```
<div style="display: none;">Not visible</div> <div style="visibility: hidden;">Hidden but space reserved</div>
```

Live Demo:

Display: none - element completely removed

Visibility: hidden - space is reserved

Example 8: Display Table

Create table-like layouts without using <table> elements.

CSS Code:

```
/* Table display */ .table { display: table; width: 100%; } .row { display: table-row; } .cell { display: table-cell; border: 1px solid #00bcd4; padding: 12px; background: #e0f7fa; } /* Useful for older browser compatibility */ /* Modern projects use flexbox/grid instead */
```

HTML Usage:

```
<div style="display: table; width: 100%;"> <div style="display: table-row;"> <div style="display: table-cell; padding: 12px;">Cell 1</div> </div> </div>
```

Live Demo:

Cell 1

Cell 2

Cell 3

Cell 4

Cell 5

Cell 6

Example 9: Responsive Display Changes

Change display values for different screen sizes.

CSS Code:

```
/* Desktop - flex layout */ .container { display: flex; gap: 20px; } .sidebar { width: 25%; } .content { width: 75%; } /* Tablet - single column */ @media (max-width: 768px) { .container { display: flex; flex-direction: column; } .sidebar { width: 100%; } .content { width: 100%; } } /* Mobile - grid for cards */ @media (max-width: 480px) { .card-grid { display: grid; grid-template-columns: 1fr; } }
```

HTML Usage:

```
<div style="display: flex; gap: 20px;"> <aside style="width: 25%;">Sidebar</aside> <main style="width: 75%;">Content</main> </div>
```

Note:

Responsive display changes adapt layouts to different screen sizes. Desktop shows flex with columns, mobile shows single column.

Best Practices

1. Use Flexbox for 1D Layouts

```
/* Navigation, buttons in a row */ .nav { display: flex; gap: 15px; align-items: center; }
```

2. Use Grid for 2D Layouts

```
/* Complex page layouts */ .page { display: grid; grid-template-columns: 1fr 3fr 1fr; grid-template-rows: auto 1fr auto; gap: 20px; }
```

3. Avoid Table Display

```
/* X Old approach */ display: table; /* Y Modern approach */ display: flex; /* or */ display: grid;
```

4. Strategic Display: None

```
/* Hide on specific screens */ @media (max-width: 768px) { .desktop-only { display: none; } } @media (min-width: 769px) { .mobile-only { display: none; } }
```

5. Combine Display Properties

```
/* Flex container with responsive items */ .container { display: flex; flex-wrap: wrap; } .item { flex: 1 1 200px; flex: grow, min-width */ }
```

Tip: Display: flex and display: grid are the modern standards. Use them instead of floats and table-based layouts for cleaner, more maintainable code.

Common Issues & Solutions

Issue: Inline Elements Won't Size

Problem:

```
span { width: 200px; /* Ignored */ height: 100px; /* Ignored */ }
```

Solution:

```
span { display: inline-block; width: 200px; /* Now works */ height: 100px; /* Now works */ }
```

Issue: Vertical Align Not Working

Problem:

```
div { vertical-align: middle; /* Doesn't work */ }
```

Solution:

```
div { display: flex; align-items: center; /* Works */ justify-content: center; }
```

Issue: Elements Wrapping Unexpectedly

Problem:

```
.container { display: flex; /* Items wrap unexpectedly */ }
```

Solution:

```
.container { display: flex; flex-wrap: nowrap; /* Prevent wrapping */ /* or */ flex-wrap: wrap; /* Allow wrapping */ }
```

Summary

- Use **display: block** for full-width elements
- Use **display: inline** for text-level elements
- Use **display: inline-block** for both inline flow and sizing
- Use **display: flex** for 1D layouts (rows or columns)
- Use **display: grid** for 2D layouts (rows and columns)
- Use **display: none** to remove elements completely
- Use **visibility: hidden** to hide but preserve space
- Avoid **display: table** in modern projects
- Combine display with responsive queries for mobile optimization
- Test display behavior across screens and devices
- Choose display value based on layout needs, not browser support